第五章 方法、泛型和特征

5.1 方法（Method）

- 方法指的是“成员函数”，是**结构体**或**枚举**的成员函数
- C++：

```cpp
class Rectangle {
private:
    int width;
    int height;

public:
    Rectangle(int w, int h) : width(w), height(h) {}

    // 成员函数 - 相当于 Rust 的方法
    int area() const {
        return width * height;
    }

    bool canHold(const Rectangle& other) const {
        return width > other.width && height > other.height;
    }
};
```

- Rust：

```rust
struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    // 关联函数 - 相当于构造函数
    fn new(width: u32, height: u32) -> Self {
        Rectangle { width, height }
    }

    // 方法 - 相当于 const 成员函数
    fn area(&self) -> u32 {
        self.width * self.height
    }

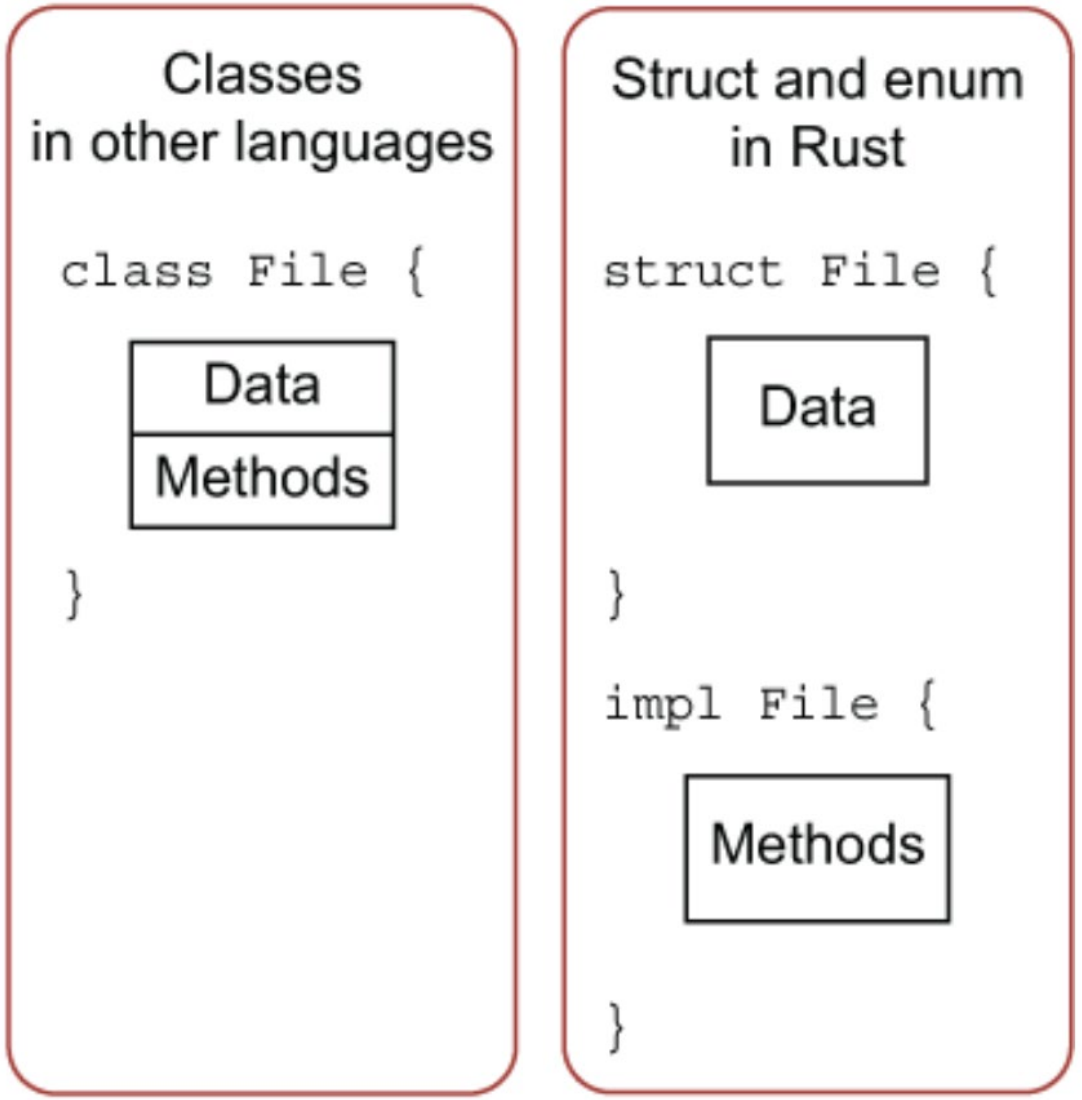
    // 另一个方法
    fn can_hold(&self, other: &Rectangle) -> bool {
        self.width > other.width && self.height > other.height
    }
}
```

在“impl”中定义结构体的方法

大学计

5.1 方法（Method）

- 其它语言中所有定义都在 class 中，但是 Rust 的对象定义和方法定义是分离的



5.1 方法（Method）

```
struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }
}

fn main() {
    let rect1 = Rectangle { width: 30, height: 50 };

    println!(
        "The area of the rectangle is {} square pixels.",
        rect1.area()
    );
}
```

定义“Rectangle”结构体

1、在结构体定义基础上，定义“area”成员函数
2、输入参数&self可以理解为“this”

和C++类成员函数一样的调用形式

5.1 方法——self、&self 、 &mut self、Self

```rust
impl Rectangle {
    // 1. 不可变借用 - 只读访问
    fn area(&self) -> u32 {
        self.width * self.height
    }

    // 2. 可变借用 - 可以修改结构体
    fn double_size(&mut self) {
        self.width *= 2;
        self.height *= 2;
    }

    // 3. 获取所有权 - 消费该实例
    fn consume(self) -> String {
        format!("消耗了 {}x{} 的矩形", self.width, self.height)
    }
}
```

&self：不可变借用—只读访问

&mut self：可变借用

self：可变、直接获取对象所有权，原对象不能再使用

Self：指代被实现方法的结构体类型，等同于“Rectangle”

5.1 方法——多种形式

- C++：构造函数、访问器、一般成员函数
- 1、Rust的构造函数包含在“关联函数”定义中：
  - 定义在 impl 中且没有 self 输入参数的函数被称之为关联函数

```rust
struct Point {
    x: f64,
    y: f64,
}

impl Point {
    // 关联函数 - 没有 self 参数
    fn new(x: f64, y: f64) -> Self {
        Point { x, y }
    }
```

1、函数名通常写为new（自定义）
2、没有self输入参数，但返回对象类型

3、调用时不能用 “.” 而是“：：”

```rust
fn main() {
    // 使用 :: 语法调用关联函数
    let point1 = Point::new(3.0, 4.0);
```

5.1 方法——多种形式

- 2、访问器：允许方法名跟结构体的成员变量名相同，通常用于访问器，在私有访问权限中使用

```
pub struct Rectangle {
    width: u32,
    pub height: u32,
}

impl Rectangle {
    pub fn new(width: u32, height: u32) -> Self {
        Rectangle { width, height }
    }
    pub fn width(&self) -> u32 {
        return self.width;
    }
    pub fn height(&self) -> u32 {
        return self.height;
    }
}
```

1、成员变量名可以和方法名相同
2、xx.width()调用方法；xx.width直接访问成员变量

为什么要多此一举?

5.1 方法——多种形式

- 3、一般成员函数：带&self参数，可以有其他不同参数输入

```
impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }

    fn can_hold(&self, other: &Rectangle) -> bool {
        self.width > other.width && self.height > other.height
    }
}

fn main() {
    let rect1 = Rectangle { width: 30, height: 50 };
    let rect2 = Rectangle { width: 10, height: 40 };
    let rect3 = Rectangle { width: 60, height: 45 };

    println!("Can rect1 hold rect2? {}", rect1.can_hold(&rect2));
    println!("Can rect1 hold rect3? {}", rect1.can_hold(&rect3));
}
```



5.1 方法——和C++的主要区别

- Rust 的方法定义在独立的 impl 块中，而不是在类型定义内部
- Rust 通过明确的 self 参数来表明接收者，而不是隐式的 this
- Rust 的所有权系统使得方法调用时的权限非常明确

5.2 泛型——函数

- 需求：用同一函数处理不同输入类型但计算步骤相同。例如两个数的加法，无论是整数还是浮点数，甚至是自定义类型，都能进行支持
  - C++中的函数重载？

没有泛型、重载的情况下要写多个函数

```rust
fn add_i8(a:i8, b:i8) -> i8 {
    a + b
}
fn add_i32(a:i32, b:i32) -> i32 {
    a + b
}
fn add_f64(a:f64, b:f64) -> f64 {
    a + b
}

fn main() {
    println!("add i8: {}", add_i8(2i8, 3i8));
    println!("add i32: {}", add_i32(20, 30));
    println!("add f64: {}", add_f64(1.23, 1.23));
}
```

函数重载：同名函数，但输入输出参数类型可以不一样

```cpp
#include <iostream>

// 重载 1：针对 int 类型
int add(int a, int b) {
    std::cout << "Adding integers" << std::endl;
    return a + b;
}

// 重载 2：针对 double 类型
double add(double a, double b) {
    std::cout << "Adding doubles" << std::endl;
    return a + b;
}

// 重载 3：参数数量不同
int add(int a, int b, int c) {
    return a + b + c;
}

int main() {
    add(1, 2);      // 调用重载 1
    add(1.0, 2.0);  // 调用重载 2
    add(1, 2, 3);   // 调用重载 3
    return 0;
}
```

5.2 泛型——函数

- 使用“T”来表示泛型参数，即“可以是任意参数类型”
- 示例1:

```
fn add<T>(a:T, b:T) -> T {
    a + b
}

fn main() {
    println!("add i8: {}", add(2i8, 3i8));
    println!("add i32: {}", add(20, 30));
    println!("add f64: {}", add(1.23, 1.23));
}
```

1、任意类型的a和b的输入，返回相同类型
2、编译报错，需要后续学习trait特征
3、任意类型可以直接做加法吗?

- 示例2:

```
fn largest<T>(list: &[T]) -> T {
```

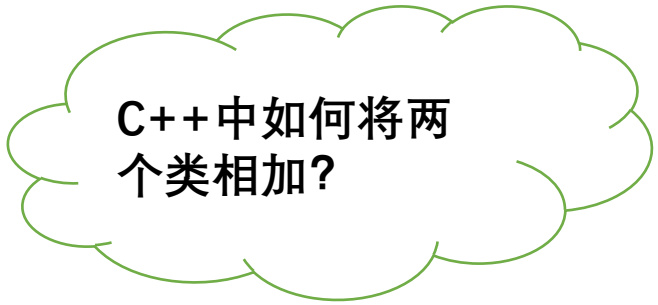
1、从列表中找出最大的值，其中列表中的元素类型为 T
2、largest<T> 对泛型参数 T 进行了声明，然后才在函数参数中进行使用该泛型参数 list: &[T]

5.2 泛型——函数

- 针对示例1的编译问题，需要做如下修改：

```
fn add<T: std::ops::Add<Output = T>>(a:T, b:T) -> T {
    a + b
}
```

- 这段代码中的 T: std::ops::Add<Output = T> 是一个 **trait 约束**，它包含两层重要含义：
  - **1. 类型 T 必须支持 + 运算符**
  - **2. 加法结果必须是同类型 T**



5.2 泛型——结构体

- 结构体中使用泛型：

```
struct Point<T> {
    x: T,
    y: T,
}

fn main() {
    let integer = Point { x: 5, y: 10 };
    let float = Point { x: 1.0, y: 4.0 };
}
```

定义了一个坐标点 Point，它可以存放任何类型的坐标值

- 提前声明，使用泛型参数之前必需要进行声明 Point<T>，接着就可以在结构体的字段类型中使用 T 来替代具体的类型
- x 和 y 定义成相同的类型T后，调用时不能不一样比如：

```
fn main() {
    let p = Point{x: 1, y :1.1};
}
```

1、一个是整形一个是浮点是不匹配的！需要理解T的含义
2、当把 1 赋值给 x 时，变量 p 的 T 类型就被确定为整数类型

5.2 泛型——结构体

- 如果想让 x 和 y 既能类型相同，又能类型不同，就需要使用不同的泛型参数

```
struct Point<T,U> {
    x: T,
    y: U,
}
fn main() {
    let p = Point{x: 1, y :1.1};
}
```

提前声明T、U两个泛型参数

5.2 泛型——方法

- 方法中使用泛型（1）：

```
struct Point<T> {
    x: T,
    y: T,
}

impl<T> Point<T> {
    fn x(&self) -> &T {
        &self.x
    }
}

fn main() {
    let p = Point { x: 5, y: 10 };

    println!("p.x = {}", p.x());
}
```

1、结构体定义时
申明泛型参数T

2、定义方法时在
impl<T>申明泛型参数T

5.2 泛型——方法

- 方法中使用泛型（2）：为具体的泛型类型实现方法

```
impl Point<f32> {
    fn distance_from_origin(&self) -> f32 {
        (self.x.powi(2) + self.y.powi(2)).sqrt()
    }
}
```

1、不仅能定义基于 T 的方法，还能针对特定的具体类型
2、这里仅针对f32即单精度浮点输入类型声明了方法

5.2 泛型——方法

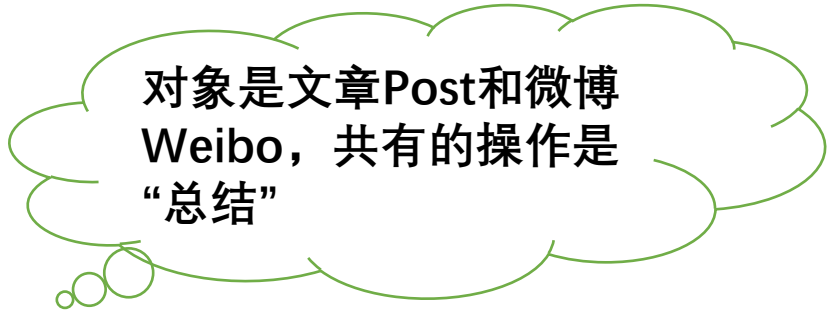
- 针对示例1的编译问题，需要做如下修改：

```
fn add<T: std::ops::Add<Output = T>>(a:T, b:T) -> T {
    a + b
}
```

5.3 特征trait

- 需求：**操作**和**对象**的解耦。
  - 例如：文件操作主要包含四个：open、write、read、close，但操作对象可以是主存，可以是辅存，还可以是管道等等。针对不同对象的一套同样的操作，是否需要单独实现一套代码？
- 特征trait，定义了**一组可以被共享的操作，只要实现了特征，你就能使用这组操作**。
  - 不同的类型具有相同的操作，那么我们就可以定义一个特征，然后为这些类型实现该特征。**定义特征**是把一些方法组合在一起，目的是定义一个实现某些目标所必需的操作的集合

5.3 特征



- 例如，有文章 Post 和微博 Weibo 两种内容载体，而我们想对相应的内容进行总结，也就是无论是文章内容，还是微博内容，都可以在某个时间点进行总结，那么总结这个操作就是共享的，因此可以用特征来定义：

```
pub trait Summary {
    fn summarize(&self) -> String;
}
```

- 1、使用 trait 关键字来声明一个特征，Summary 是特征名，pub是“public”
- 2、在大括号中定义了该特征的所有方法，在这个例子中是： fn summarize(&self) -> String
- 3、特征只定义行为看起来是什么样的，而不定义行为具体是怎么样的。因此，我们只定义特征方法的申明，而不进行实现，此时方法签名结尾是 ;，而不是一个 {}。有点类似于“虚函数”

5.3 特征——和C++虚函数的区别与联系

- 联系：都是一种“多态”的实现，允许不同类型的对象对同一消息做出不同的响应。

```
// 必须通过继承建立类型层次
class Animal {
public:
    virtual void speak() = 0;
};

class Dog : public Animal {  // Dog "是一个" Animal
public:
    void speak() override { cout << "Woof!" << endl; }
};

// 使用：必须通过基类指针/引用
Animal* animal = new Dog();
animal->speak();
```

基类为animal，有一个speak虚函数。即“是动物就会叫”

派生“狗”类的时候，需要对虚函数进行实现

用基类指针调用派生类对象成员函数

5.3 特征——和C++虚函数的区别与联系

- **C++ 虚函数 "他是一个" (is-a) 关系**

```cpp
// 必须通过继承建立类型层次
class Animal {
public:
    virtual void speak() = 0;
};

class Dog : public Animal {  // Dog "是一个" Animal
public:
    void speak() override { cout << "Woof!" << endl; }
};

// 使用：必须通过基类指针/引用
Animal* animal = new Dog();
animal->speak();
```

- **Rust trait - "他能够" (can-do) 关系**

```rust
// 定义能力接口，与类型层次无关
trait Speak {
    fn speak(&self);
}

struct Dog; // 普通结构体，不继承任何东西
struct Cat;
struct Human;

// 为不同类型实现相同能力
impl Speak for Dog {
    fn speak(&self) {
        println!("Woof!");
    }
}

impl Speak for Cat {
    fn speak(&self) {
        println!("Meow!");
    }
}

// 使用：任何实现了 Speak 的类型都可以
fn make_speak<T: Speak>(speaker: &T) {
    speaker.speak();
}
```

5.3 特征——为类型实现特征

- 因为“特征”只定义行为看起来是什么样的，因此我们需要为类型实现具体的特征，定义行为具体是怎么样的。
  - 首先来为 Post 和 Weibo 实现 Summary 特征

```rust
pub trait Summary {
    fn summarize(&self) -> String;
}
pub struct Post {
    pub title: String, // 标题
    pub author: String, // 作者
    pub content: String, // 内容
}

impl Summary for Post {
    fn summarize(&self) -> String {
        format!("文章{}, 作者是{}", self.title, self.author)
    }
}

pub struct Weibo {
    pub username: String,
    pub content: String
}

impl Summary for Weibo {
    fn summarize(&self) -> String {
        format!("{}发表了微博{}", self.username, self.content)
    }
}
```

1、impl Summary for Post，读作“为 Post 类型实现 Summary 特征”，然后在 impl 的花括号中实现该特征的具体方法
2、为不同的对象定义具体的行为
3、使用时直接调用操作

```rust
fn main() {
    let post = Post{title: "Rust语言简介".to_string(),a
    let weibo = Weibo{username: "sunface".to_string()

    println!("{}",post.summarize());
    println!("{}",weibo.summarize());
}
```

5.3 特征——孤儿规则

- 如果你想要为类型 A 实现特征 T，那么 A 或者 T 至少有一个是在当前作用域中定义的。
  - 保证其它人编写的代码不会破坏你的代码，也确保了你不会莫名其妙就破坏了风马牛不相及的代码

5.3 特征——默认实现

- 可以在特征中定义具有**默认实现**的方法，这样其它类型无需再实现该方法，或者也可以选择重写该方法（类似于类的继承）

```
pub trait Summary {
    fn summarize(&self) -> String {
        String::from("(Read more...)")
    }
}
impl Summary for Post {}

impl Summary for Weibo {
    fn summarize(&self) -> String {
        format!("{}发表了微博{}", self.username, self.cont
    }
}
```

定义Summary的trait的时候，为方法定义了“默认操作”

Post 选择了默认实现，而 Weibo 重写了该方法

```
    println!("{}",post.summarize());
    println!("{}",weibo.summarize());
```

输出

```
(Read more...)
sunface发表了微博好像微博没Tweet好用
```

5.3 特征——特征约束（Trait Bounds）

- 限制泛型类型必须实现特定的特征。基本语法：

```rust
// 泛型 T 必须实现 Display 特征
fn print_item<T: std::fmt::Display>(item: T) {
    println!("{}", item);
}

// 等价写法
fn print_item_where<T>(item: T)
where
    T: std::fmt::Display
{
    println!("{}", item);
}
```

5.3 特征——特征约束（Trait Bounds）

- 多重约束：即要求具备多个特征。基本语法：

```rust
// T 必须同时实现 Display 和 Clone
fn clone_and_print<T: Display + Clone>(item: T) {
    let cloned = item.clone();
    println!("Original: {}, Cloned: {}", item, cloned);
}

// where 子句写法
fn clone_and_print_where<T>(item: T)
where
    T: Display + Clone
{
    let cloned = item.clone();
    println!("Original: {}, Cloned: {}", item, cloned);
}
```

5.3 特征——特征约束（Trait Bounds）

- 有条件地实现方法或特征

```rust
struct Pair<T> {
    x: T,
    y: T,
}

impl<T> Pair<T> {
    fn new(x: T, y: T) -> Sel
        Self {
            x,
            y,
        }
    }
}

impl<T: Display + PartialOrd> Pair<T> {
    fn cmp_display(&self) {
        if self.x >= self.y {
            println!("The largest member is x = {}", self.x);
        } else {
            println!("The largest member is y = {}", self.y);
        }
    }
}
```

只有 T 同时实现了 Display + PartialOrd 的 Pair<T> 才可以拥有此方法

```rust
impl<T: Display> ToString for T {
    // --snip--
}
```

为任何实现了 Display 特征的类型实现 ToString 特征

5.3 特征——将特征作为函数参数或返回

- 使用特征作为函数参数，即任何实现了 Summary 特征的类型作为该函数的参数：

```
pub fn notify(item: &impl Summary) {
    println!("Breaking news! {}", item.summarize());
}
```

- 通过 impl Trait 来说明一个函数返回了一个类型，该类型实现了某个特征：

```
fn returns_summarizable() -> impl Summary {
    Weibo {
        username: String::from("sunface"),
        content: String::from(
            "m1 max太厉害了，电脑再也不会卡",
        )
    }
}
```